

Python

import os

import zipfile

Files for the complete Vite + Mastodon SCSS + Markdown Blog setup

files = {

'package.json': """{

"name": "vite-mastodon-theme-blog",

"private": true,

"version": "1.0.0",

"type": "module",

"scripts": {

"dev": "vite",

"build": "vite build",

"preview": "vite preview"

},

"dependencies": {

"marked": "^12.0.0"

},

"devDependencies": {

"sass": "^1.72.0",

"vite": "^5.1.0"

}

}

""",

'vite.config.js': """import { defineConfig } from 'vite';

import { resolve } from 'path';

export default defineConfig({

resolve: {

alias: {

'@styles': resolve(__dirname, 'src/styles'),

'@posts': resolve(__dirname, 'posts'),

},

},

css: {

preprocessorOptions: {

scss: {

api: 'modern-compiler',

```

    },
  },
},
build: {
  rollupOptions: {
    input: {
      main: resolve(__dirname, 'index.html'),
      'theme-default': resolve(__dirname, 'src/styles/reset.scss'),
      'theme-win95': resolve(__dirname, 'src/styles/win95.scss'),
      'flavour-glitch': resolve(__dirname, 'src/styles/flavours/glitch/reset.scss'),
    },
  },
},
});
""",

```

```

    'index.html': `<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Multi-Theme Tech Blog</title>
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"></script>
</body>
</html>
`,

```

```

    'posts/web4-architecture.md': `## Building Web4 Decentralized Platforms

```

Web4 architecture focuses on combining **decentralized web protocols**, autonomous execution layers, and local-first data integrity.

Key Principles

- * Zero Central Authority Dependencies**: Self-sovereign routing and execution.
- * Custom Parser Pipelines**: High-performance AST generation for custom languages.
- * Local-First Synchronization**: Immutable data structures syncing seamlessly.

```

```javascript
// Web4 Initialization Routine

```

```
function bootEngine() {
 console.log("Aura Web4 Core Engine online.");
}
bootEngine();
```

```
""",
```

'posts/compiler-pipeline.md': """"# Custom Compiler Toolchains & Parser Pipelines

Designing custom language targets requires explicit control over lexical analysis, parsing, and code generation.

## Pipeline Breakdown

1. **Lexical Analysis (Tokenizer):** Converts raw string inputs into tokens.
2. **Parser (ANTLR/Yacc):** Builds Abstract Syntax Trees (AST).
3. **IR Generation:** Intermediate representation transformation.
4. **Target Emission:** Output assembly or web runtime binaries.

Bash

```
Example compilation invocation
aura-cli build main.aura --target web4-wasm
```

```
""",
```

'src/styles/reset.scss': """"// =====

```
// Web App Core CSS Reset
// =====
*,
*::before,
*::after {
 box-sizing: border-box;
 margin: 0;
 padding: 0;
}
html, body {
```

```
height: 100%;
width: 100%;
}
body {
line-height: 1.6;
background-color: var(--app-bg, #111827);
color: var(--app-text, #f3f4f6);
font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
padding: 2rem;
}
header {
margin-bottom: 2rem;
padding-bottom: 1rem;
border-bottom: 1px solid #374151;
display: flex;
justify-content: space-between;
align-items: center;
}
.theme-selector button {
background: #374151;
color: #fff;
border: none;
padding: 0.5rem 1rem;
margin-left: 0.5rem;
border-radius: 4px;
cursor: pointer;
}
.card {
background: #1f2937;
padding: 1.5rem;
border-radius: 8px;
margin-bottom: 1.5rem;
}
code {
background: #374151;
padding: 0.2rem 0.4rem;
border-radius: 4px;
}
pre {
background: #000;
padding: 1rem;
border-radius: 6px;
overflow-x: auto;
```

```
margin: 1rem 0;
}
""",
```

```
'src/styles/flavours/glitch/reset.scss': """/ =====
```

```
// Custom Flavour Extended Reset & Utilities
// =====
@import '../reset.scss';
body {
 --app-bg: #090a0f;
 --app-text: #00ffcc;
 font-family: 'Courier New', Courier, monospace;
}
.card {
 background: #121824;
 border: 1px solid #00ffcc;
 box-shadow: 0 0 10px rgba(0, 255, 204, 0.2);
}
.theme-selector button {
 background: #00ffcc;
 color: #000;
 font-weight: bold;
}
""",
```

```
'src/styles/win95.scss': """/ =====
```

```
// Windows 95 Retro Theme
// =====
$w95-teal: #008080;
$w95-silver: #c0c0c0;
$w95-blue: #000080;
$w95-border-light: #ffffff;
$w95-border-dark: #808080;
$w95-border-black: #000000;
$w95-text: #000000;
body {
 background-color: $w95-teal !important;
}
```

```
font-family: "MS Sans Serif", "Microsoft Sans Serif", Tahoma, sans-serif !important;
color: $w95-text !important;
padding: 1.5rem;
}
header {
background: linear-gradient(90deg, $w95-blue, #1084d0) !important;
color: #ffffff !important;
padding: 6px 12px !important;
border-top: 2px solid $w95-border-light;
border-left: 2px solid $w95-border-light;
border-right: 2px solid $w95-border-dark;
border-bottom: 2px solid $w95-border-dark;
}
.theme-selector button {
background-color: $w95-silver !important;
color: $w95-text !important;
border-top: 2px solid $w95-border-light !important;
border-left: 2px solid $w95-border-light !important;
border-right: 2px solid $w95-border-dark !important;
border-bottom: 2px solid $w95-border-dark !important;
font-weight: bold;
padding: 2px 8px;
&:active {
border-top: 2px solid $w95-border-dark !important;
border-left: 2px solid $w95-border-dark !important;
border-right: 2px solid $w95-border-light !important;
border-bottom: 2px solid $w95-border-light !important;
}
}
.card {
background-color: $w95-silver !important;
border-top: 2px solid $w95-border-light !important;
border-left: 2px solid $w95-border-light !important;
border-right: 2px solid $w95-border-dark !important;
border-bottom: 2px solid $w95-border-dark !important;
box-shadow: 1px 1px 0px $w95-border-black;
border-radius: 0 !important;
color: $w95-text !important;
margin-bottom: 1.5rem;
padding: 1rem;
}
pre {
background: #ffffff !important;
```

```

color: #000000 !important;
border-top: 2px solid $w95-border-dark !important;
border-left: 2px solid $w95-border-dark !important;
border-right: 2px solid $w95-border-light !important;
border-bottom: 2px solid $w95-border-light !important;
padding: 0.5rem;
}
""",

```

```

'src/scripts/themeLoader.js': ""const themeModules = {

default: () => import('@styles/reset.scss'),
glitch: () => import('@styles/flavours/glitch/reset.scss'),
win95: () => import('@styles/win95.scss'),
};
export async function setAppTheme(themeName) {
if (!themeModules[themeName]) themeName = 'win95';
await themeModules[themeName];
document.documentElement.setAttribute('data-theme', themeName);
}
""",

```

```

'src/scripts/blogLoader.js': ""import { parse } from 'marked';

const postFiles = import.meta.glob('/posts/*.md', { query: '?raw', eager: true });
export function getAllPosts() {
return Object.keys(postFiles).map((filepath) => {
const rawContent = postFiles[filepath].default;
const slug = filepath.replace('/posts/', '').replace('.md', '');

```

```

const titleMatch = rawContent.match(/^#\s+(.*)$/m);
const title = titleMatch ? titleMatch[1] : slug;

```

```

const htmlContent = parse(rawContent);

```

```

return {
 slug,

```

```
 title,
 content: htmlContent,
 };

 });
 }
 """,
 ,
```

```
'src/main.js': ""import { getAllPosts } from './scripts/blogLoader';
```

```
import { setAppTheme } from './scripts/themeLoader';
const activeTheme = localStorage.getItem('blog-theme') || 'win95';
setAppTheme(activeTheme);
const posts = getAllPosts();
const appContainer = document.getElementById('app');
appContainer.innerHTML = `
window.changeTheme = (themeName) => {
 localStorage.setItem('blog-theme', themeName);
 setAppTheme(themeName);
 window.location.reload();
};
""`,
 ,
```

```
'README.md': ""# Vite + Mastodon SCSS Multi-Theme Blog Repo
```

A complete, zero-backend blog setup combining Vite, Markdown parsing (marked), and Mastodon/Glitch/Win95 dynamic SCSS themes.

## Getting Started

1. Install dependencies:

```
Bash
npm install
```

2. Start local development server:

```
Bash
npm run dev
```

3. Build for production:

```
Bash
npm run build
```

## Adding Blog Posts

Add .md files into the /posts folder. Vite automatically discovers, parses, and renders them.

```
""
}
zip_filename = "vite-mastodon-blog-repo.zip"
with zipfile.ZipFile(zip_filename, 'w', zipfile.ZIP_DEFLATED) as zipf:
 for filepath, content in files.items():
 zipf.writestr(filepath, content)
print(f"Created ZIP archive: {zip_filename}")
```

```
```text?code_stdout&code_event_index=1
Created ZIP archive: vite-mastodon-blog-repo.zip
```

The entire integrated **Vite + SCSS Themes + Markdown Blog repository** is packaged into a ZIP file.



vite-mastodon-blog-repo
ZIP
Open

What's included in vite-mastodon-blog-repo.zip:

- **package.json & vite.config.js** — Pre-configured with Sass and marked dependencies, plus multi-theme SCSS build entries.
- **posts/** — Sample Markdown files (web4-architecture.md, compiler-pipeline.md) ready for parsing.
- **src/styles/** — Full theme system:
 - reset.scss (Default modern web reset)
 - flavours/glitch/reset.scss (Glitch theme override)
 - win95.scss (Retro Windows 95 theme)
- **src/scripts/** — blogLoader.js (Vite Markdown glob parser) & themeLoader.js (Dynamic runtime switcher).
- **src/main.js & index.html** — Application entry point with dynamic theme switching toolbar.

How to Run Locally

1. Extract the downloaded archive.
2. Open terminal in the directory and run:
Bash
npm install
npm run dev
3. Open <http://localhost:5173> to test the blog and switch between themes live!